

Universidad Pontificia Comillas

ICAI

CUADERNO DE TRABAJO

Aprendizaje por refuerzo

Lydia Ruiz Martínez, Pablo Tuñón Laguna, Alberto Velasco Rodríguez

**APLICACIÓN DE CONCEPTOS GENERALES
DE RL**



Curso 2025-2026

Índice

1. Navegación con tile coding	2
1.1. Representación y generalización	2
1.2. Aproximación lineal e interpretación del gradiente	2
1.3. Recompensa en <code>step()</code> : diseño, escala y riesgos	2
1.4. Funciones <code>get_q_values</code> y <code>update</code>	3
1.5. Exploración de ϵ	3
1.6. Parámetros	3
1.7. Comparativa SARSA vs Q-learning	3
2. Operación en fábrica	6
2.1. Descripción de los entornos	6
2.2. Representación del estado	6
2.3. Modelos alternativos probados	7
2.4. Diseño de la recompensa	8
2.5. Agente y algoritmo	9
2.6. Comparativa de hiperparámetros y entrenamiento	9
2.7. Resultados	10
2.8. Conclusión	11

1. Navegación con tile coding

Esta sección documenta el diseño, las decisiones y los resultados de la práctica de navegación con aproximación lineal y *tile coding*. El objetivo es aprender una política que desplace al agente en un mapa de 10 m por 10 m hasta una zona objetivo evitando colisiones, usando acciones discretas de desplazamiento cardinal con paso de 0.25 m. Se parte de una observación cruda $(x, y, \text{collision}, \text{target})$ y se construye una representación dispersa mediante tiles solapados.

1.1. Representación y generalización

El estado continuo no se presta a una tabla de valores, así que se emplea *tile coding* con varias rejillas solapadas. Cada rejilla activa exactamente un índice, por lo que el vector de entrada $\mathbf{x}(s)$ tiene exactamente n_{tilings} de unos y el resto ceros. La normalización por anchura y altura de tile desacopla la escala del mapa de la discretización. Se aplican desplazamientos distintos por rejilla con paso relativo (3, 1) en los ejes para romper alineamientos regulares y evitar patrones repetitivos que degradan la generalización. Con $n_{\text{tiles_width}} = 10$, $n_{\text{tiles_height}} = 10$ y $n_{\text{tilings}} = 8$ se obtiene un compromiso útil: resolución suficiente en pasillos y solapamiento para transferir experiencia entre posiciones cercanas.

Los pesos se almacenan como una matriz $\mathbf{weights} \in \mathbb{R}^{\text{num_actions} \times \text{feature_size}}$. Esta forma evita ambigüedades de indexado y permite sumar de manera vectorizada en las columnas correspondientes a los índices activos.

1.2. Aproximación lineal e interpretación del gradiente

Se aproxima $Q(s, a)$ con un modelo lineal por acción,

$$Q(s, a) \approx \mathbf{w}_a^\top \mathbf{x}(s),$$

donde $\mathbf{x}(s)$ es binaria y dispersa. La derivada de Q respecto a $\mathbf{w}_a$ es exactamente $\mathbf{x}(s)$, así que el gradiente está implícito al actualizar únicamente las coordenadas activas. En cada transición se modifican n_{tilings} pesos de la fila asociada a la acción ejecutada. Esto aconseja escalar la tasa de aprendizaje con el número de rejillas; como regla práctica, $\alpha \approx 0,1/n_{\text{tilings}}$ ofrece magnitudes razonables.

1.3. Recompensa en step(): diseño, escala y riesgos

El cálculo de la recompensa se integra en `step()` con tres casos:

$$r = \begin{cases} +100, & \text{si el agente entra en la zona objetivo,} \\ -100, & \text{si hay colisión,} \\ -c - k(|x - c_x| + |y - c_y|), & \text{en otro caso,} \end{cases}$$

donde (c_x, c_y) es el centro de la zona objetivo. El término L1 genera una pendiente constante hacia la meta y evita efectos indeseados de la norma L2 (gradientes pequeños lejos del objetivo y posibles ganancias positivas al cuadrar). La escala de los

extremos y del shaping es crítica: si los terminales son demasiado grandes dominan la señal y aparecen trayectorias absurdas; si c y k son demasiado pequeños, falta guía intermedia. Una horquilla inicial razonable es $c \in [0,05, 0,1]$ y $k \in [0,005, 0,02]$. Para atascos o colisiones sistemáticas se puede reducir $|100|$, subir c para penalizar pasos inútiles o ajustar k para hacer más pronunciada la pendiente.

Cuando una acción intenta cruzar una pared, conviene no mover la posición y terminar el episodio por colisión. La comprobación de llegada se hace sobre la posición efectiva tras validar colisiones.

1.4. Funciones `get_q_values` y `update`

`get_q_values` suma los pesos en los índices activos para cada acción. De forma vectorizada,

$$\text{q_values} = \text{weights[:, idx].sum(axis=1)}.$$

En SARSA (semi-gradiente):

$$\delta = \begin{cases} r - Q(s, a), & \text{si termina,} \\ r + \gamma Q(s', a') - Q(s, a), & \text{en otro caso,} \end{cases} \quad \mathbf{w}_a \leftarrow \mathbf{w}_a + \alpha \delta \mathbf{x}(s).$$

En Q-learning se sustituye $Q(s', a')$ por $\max_{a'} Q(s', a')$. En ambos casos se actualizan solo las coordenadas activas.

1.5. Exploración de ϵ

La política de comportamiento es ϵ -greedy con decaimiento exponencial. Tres parámetros la gobiernan: fracción de episodios a partir de la cual empieza el decaimiento, razón de decaimiento y valor mínimo. Iniciar muy pronto o decaer agresivo cierra la exploración; iniciar muy tarde ralentiza la consolidación. En los ensayos fue equilibrado empezar entre el 50–70 % de episodios, con razón en $[0,95, 0,99]$ y mínimo en $[0,02, 0,05]$, de modo que al final la política sea casi determinista (como exige la evaluación).

1.6. Parámetros

Se fijaron $n_{\text{tiles_width}} = 10$, $n_{\text{tiles_height}} = 10$, $n_{\text{tilings}} = 8$, zona objetivo $(2,5, 8, 1,0, 2,0)$, $\gamma = 0,99$ y 10,000 episodios. Los hiperparámetros más sensibles fueron α y el calendario de ϵ : un decaimiento excesivo degradó por cierre prematuro; posponer demasiado el inicio enlentece la convergencia. El refactor incluyó centralizar código, corregir la forma de `weights` a matriz bidimensional y exponer parámetros antes embebidos.

1.7. Comparativa SARSA vs Q-learning

Se entrenaron ambos agentes con los mismos tiles, $\gamma = 0,99$, 10,000 episodios, y un calendario de ϵ con decaimiento tardío. En este entorno determinista, Q-learning

suele consolidar trayectorias más rápidas; SARSA tiende a políticas algo más conservadoras si hay estocasticidad. Las figuras siguientes muestran las métricas de cada agente:

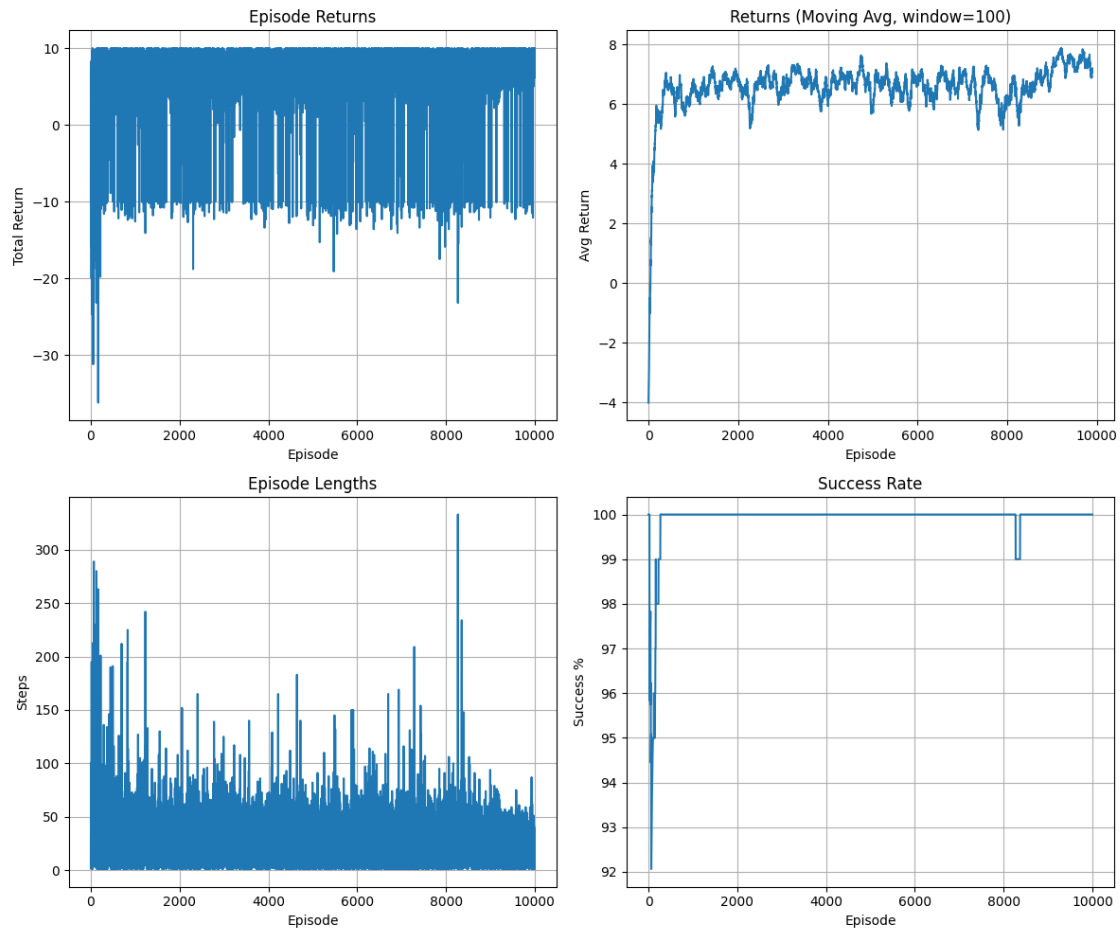


Figura 1: Q-learning: retornos, media móvil (100), longitudes y éxito.

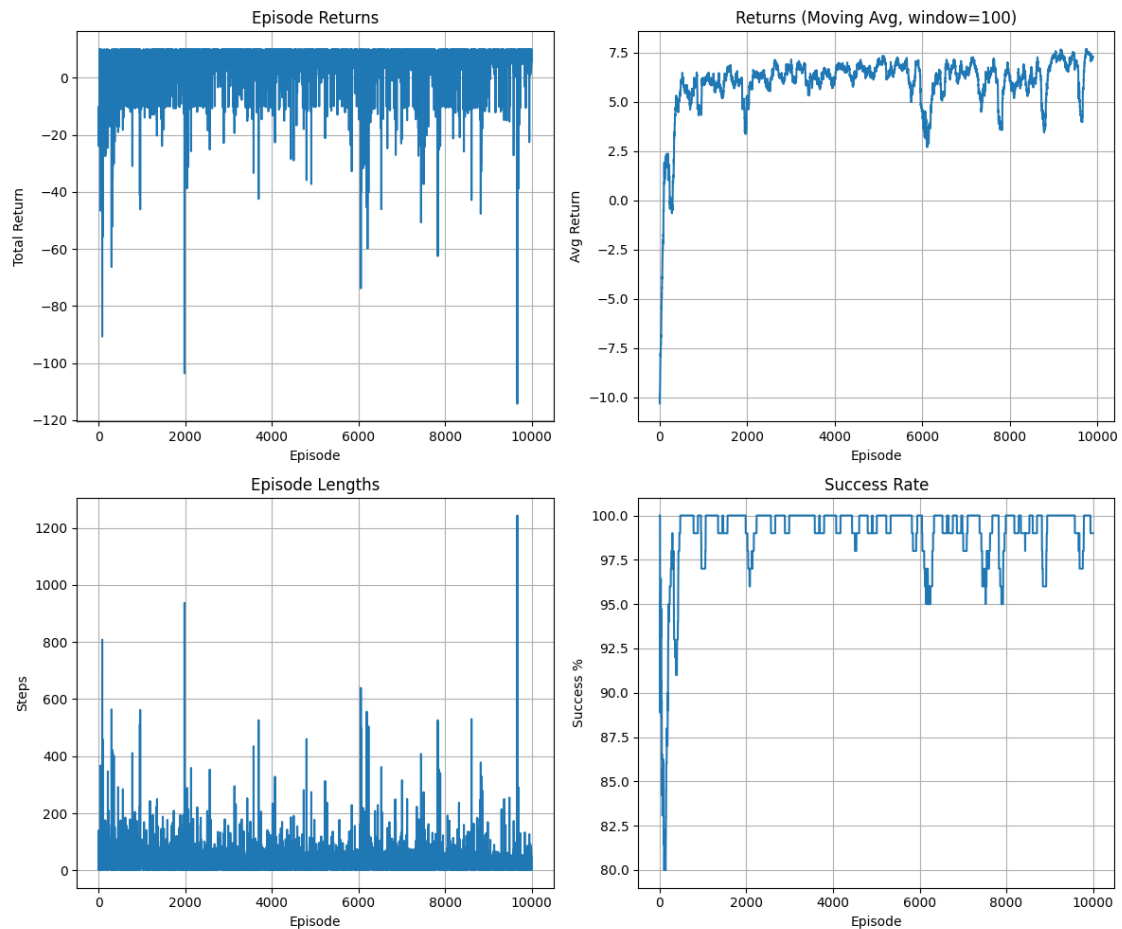


Figura 2: SARSA: retornos, media móvil (100), longitudes y éxito.

En los resultados que obtuvimos, ambos alcanzan tasas de éxito muy altas. Q-learning tiende a converger algo más rápido en la media móvil y muestra longitudes medianas ligeramente menores en los últimos episodios; SARSA exhibe mayor variabilidad episódica pero una media móvil muy estable al final. Estas diferencias son coherentes con el objetivo temporal *off-policy* de Q-learning frente a la naturaleza *on-policy* de SARSA.

2. Operación en fábrica

En esta sección se aborda la resolución de los entornos de almacén mediante aprendizaje por refuerzo con aproximación lineal y *tile coding*. El agente opera en un espacio continuo de $10\text{ m} \times 10\text{ m}$, con obstáculos fijos (estanterías) y objetos a recoger y/o entregar según el entorno.

2.1. Descripción de los entornos

- **Entorno 1:** Los objetos están en posiciones fijas sobre las estanterías. El objetivo consiste en que el agente se aproxime a cualquiera de ellos y lo recoja. El episodio termina con éxito al recoger un objeto, o con fracaso si hay colisión con paredes o estanterías.
- **Entorno 2:** Los objetos siguen en posiciones fijas, pero el objetivo es entregar uno de ellos en la zona verde de recogida. El agente debe primero recoger un objeto y después depositarlo correctamente en el área de entrega. El episodio termina con éxito al soltar el objeto en la zona de entrega, o con fracaso si hay colisión o si se suelta el objeto fuera del área.
- **Entorno 3:** Similar al entorno 2, pero la posición de los objetos en las estanterías cambia aleatoriamente en cada episodio, lo que añade mayor variabilidad y dificultad al problema. El objetivo sigue siendo recoger y entregar un objeto en la zona de entrega.

2.2. Representación del estado

Para todos los entornos, se emplea *tile coding* como técnica de representación del estado. La observación del entorno incluye la posición del agente, las posiciones de los objetos, y banderas de colisión y entrega. El estado se normaliza y se codifica en mosaicos solapados, lo que permite generalizar entre posiciones cercanas y facilita el aprendizaje en espacios continuos.

La información utilizada para la codificación varía según el entorno:

- **Entorno 1:** Se utiliza únicamente la posición normalizada del agente y si porta un objeto.
- **Entornos 2 y 3:** Se añade información adicional sobre la dirección y distancia al objetivo (objeto o zona de entrega), permitiendo una representación más rica y adecuada para tareas de recogida y entrega.

La alternativa de representación vectorizada se ha probado, pero no ofrece buen rendimiento en estos entornos.

2.3. Modelos alternativos probados

Además del enfoque principal con SARSA(λ) y *tile coding*, se exploraron otras aproximaciones que, aunque no lograron los resultados deseados, aportan información valiosa sobre las características del problema.

2.3.1. Aproximación lineal vectorizada

Se probó una representación lineal directa utilizando las 11 variables crudas de la observación (posición del agente, posiciones de objetos, banderas de estado) como vector de *features* continuas:

$$Q(s, a) = \mathbf{w}_a^\top \mathbf{x}(s), \quad \mathbf{x}(s) \in \mathbb{R}^{11}$$

Problema identificado: La aproximación lineal asume que Q varía linealmente con las coordenadas, pero cerca de los obstáculos un desplazamiento de 0.25 m puede significar la diferencia entre éxito y colisión. Esta discontinuidad no puede capturarse con un modelo lineal sobre *features* crudas.

Resultado: Tasa de éxito inferior al 50 % en el entorno 1, con comportamiento errático cerca de las estanterías.

2.3.2. Deep Q-Network (DQN)

Se implementó una DQN con las siguientes características:

- Red neuronal de 3 capas ocultas (256-256-128-64 neuronas) con activaciones ReLU intermedias.
- *Experience replay* con buffer de 100,000 transiciones.
- *Prioritized replay* para muestrear experiencias con mayor error TD.
- Red objetivo (*target network*) actualizada cada 500 pasos.
- Soporte para retornos n -step.

Ventajas teóricas: La red neuronal puede aprender funciones no lineales arbitrarias y generalizar a estados no vistos mediante representaciones internas.

Problemas encontrados:

1. **Velocidad:** El entrenamiento resultó entre 10-20 veces más lento que TD(λ) con tiles, debido al coste computacional de las pasadas *forward/backward* y el procesamiento de *features*.
2. **Datos necesarios:** Requirió más de 50,000 episodios para alcanzar tasas de éxito comparables a las obtenidas con tiles en 3,000-30,000 episodios.
3. **Sensibilidad:** Alta sensibilidad a hiperparámetros (tasa de aprendizaje, arquitectura, tamaño de *batch*).

Resultado: La DQN no logró aprender políticas funcionales debido al coste computacional y la necesidad de ajuste fino, por lo que no quedó justificado su posterior desarrollo frente al enfoque con *tile coding*, que ofrece resultados superiores con menor complejidad.

2.3.3. Comparativa de enfoques

Modelo	Éxito Env 1	Éxito Env 2	Éxito Env 3	Episodios	Velocidad
SARSA(λ) + Tiles	95 %+	90 %+	85 %+	3k–30k	~5 it/s
SARSA(λ) + Lineal	<6 %	–	–	5k–50k	~3 it/s
DQN	20 %	–	–	50k+	~2 it/s

Cuadro 1: Comparativa de rendimiento entre los modelos probados.

2.3.4. Conclusión de la exploración

La ingeniería de *features* mediante *tile coding* con variables adicionales (dirección y distancia al objetivo) demostró ser más efectiva que aproximaciones más complejas. Esto ilustra un principio importante del aprendizaje por refuerzo: **una representación del estado bien diseñada puede superar a modelos más potentes** cuando el dominio se comprende adecuadamente.

2.4. Diseño de la recompensa

El diseño de la función de recompensa es un aspecto clave para guiar el aprendizaje del agente en ambos entornos. Se ha optado por una estructura sencilla y directa, que penaliza los pasos innecesarios y refuerza los comportamientos deseados:

- **Penalización por paso:** Cada acción realizada por el agente conlleva una penalización pequeña (en este caso, $-0,1$), incentivando trayectorias cortas y eficientes.
- **Recompensa por éxito:** Al recoger un objeto (entorno 1) o al entregar correctamente un objeto en la zona de entrega (entorno 2), el agente recibe una recompensa positiva significativa (en este caso, $+10$), que marca el objetivo principal de la tarea.
- **Penalización por fallo:** Si el agente colisiona con una pared o estantería, o si intenta realizar una acción inválida (como recoger sin estar cerca de un objeto, o soltar fuera del área de entrega), recibe una penalización fuerte (en este caso, -1), y el episodio termina.
- **Acciones inválidas:** Acciones como intentar recoger un objeto cuando ya se lleva uno, o intentar soltar sin portar objeto, reciben una penalización menor (en este caso, -1), desincentivando comportamientos erróneos pero sin terminar el episodio.

- **Penalización por *timeout*:** Si el agente supera el número máximo de pasos permitido sin completar la tarea, el episodio se trunca y se aplica una penalización intermedia (en este caso, -5), penalizando la ineficiencia y evitando que el agente aprenda a alargar los episodios innecesariamente.

Esta estructura de recompensas favorece el aprendizaje rápido y estable, ya que el agente recibe señales claras sobre qué comportamientos son deseables y cuáles debe evitar. Además, la penalización por paso ayuda a minimizar el número de acciones necesarias para completar la tarea, promoviendo trayectorias óptimas.

2.5. Agente y algoritmo

El agente implementa el algoritmo SARSA(λ), que combina la actualización temporal-diferencial con trazas de elegibilidad para acelerar la propagación de la señal de recompensa. La política de comportamiento es ϵ -greedy con decaimiento exponencial, y los hiperparámetros se ajustan para favorecer la exploración inicial y la consolidación posterior.

La aproximación lineal se realiza sobre los índices activos de los tiles, y los pesos se actualizan únicamente en las coordenadas correspondientes, siguiendo la derivada del modelo lineal.

2.6. Comparativa de hiperparámetros y entrenamiento

Aunque la estructura del entorno y el agente es similar en los dos casos, el ajuste de los hiperparámetros varía para adaptarse a la dificultad y naturaleza de cada tarea:

- **Entorno 1 (recogida):** Se emplea una tasa de aprendizaje (`learning_rate`) de 0,0125, y un número de episodios de entrenamiento reducido (3,000). El decaimiento de ϵ comienza en el 50 % de los episodios y termina en el 70 %. El valor inicial de ϵ es moderado (0,3), ya que la tarea es más sencilla y la exploración se consolida rápidamente. El factor de descuento es 0,99 y $\lambda = 0,8$.
- **Entorno 2 (recogida y entrega):** La tarea es más compleja, por lo que se reduce la tasa de aprendizaje (`learning_rate` = 0,003) para estabilizar el aprendizaje y se incrementa el número de episodios (15,000). El valor inicial de ϵ es mayor (0,5) para fomentar la exploración, y el decaimiento se inicia antes (30 %) para permitir una transición más gradual hacia la explotación. El factor de descuento es 0,995 y $\lambda = 0,7$.
- **Entorno 3 (recogida y entrega con objetos aleatorios):** Se emplea una tasa de aprendizaje (`learning_rate` = 0,005), un valor inicial de ϵ alto (0,8), y un número de episodios mayor (29,000) para compensar la mayor variabilidad en la posición de los objetos. El decaimiento de ϵ comienza en el 30 % y termina en el 70 % de los episodios. El factor de descuento es 0,99 y $\lambda = 0,8$.

Cabe destacar que, al utilizar valores altos de λ (por ejemplo, 0,8), el método SARSA(λ) se aproxima al comportamiento de Monte Carlo, ya que las actualizaciones dependen más de las recompensas futuras acumuladas a lo largo del episodio. Esto favorece la propagación eficiente de la señal de recompensa en tareas donde la secuencia óptima de acciones es larga o compleja. Estos ajustes permiten que el agente se adapte a la dificultad de cada entorno, logrando una convergencia eficiente y una política robusta en ambos casos. La diferencia principal radica en la necesidad de mayor exploración y entrenamiento en el entorno de entrega, donde la secuencia óptima requiere más pasos y una coordinación precisa entre las acciones de recogida y entrega.

2.7. Resultados

En los tres entornos, el agente SARSA(λ) con *tile coding* logra aprender políticas eficaces que resuelven la tarea en un número reducido de pasos y con alta tasa de éxito. El entrenamiento muestra una mejora progresiva en el retorno y la longitud de los episodios, y la tasa de éxito se estabiliza cerca del máximo posible. La representación por mosaicos resulta esencial para la generalización y el aprendizaje eficiente en estos escenarios.

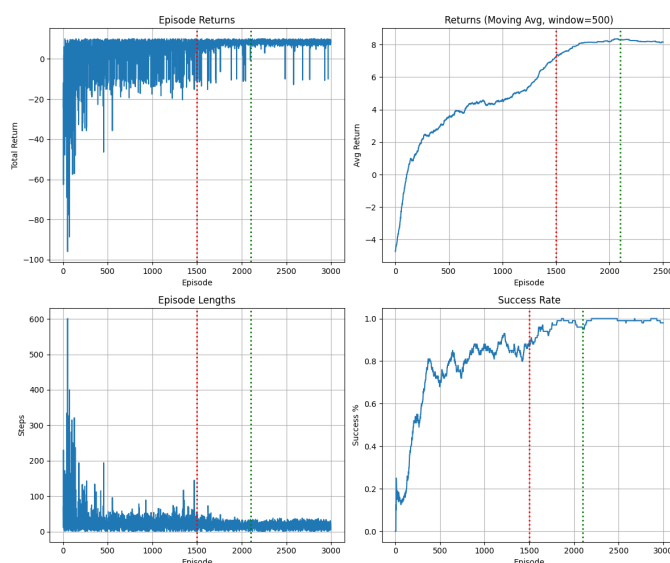


Figura 3: Métricas de entrenamiento en Entorno 1: retornos, media móvil, longitud de episodios y tasa de éxito.

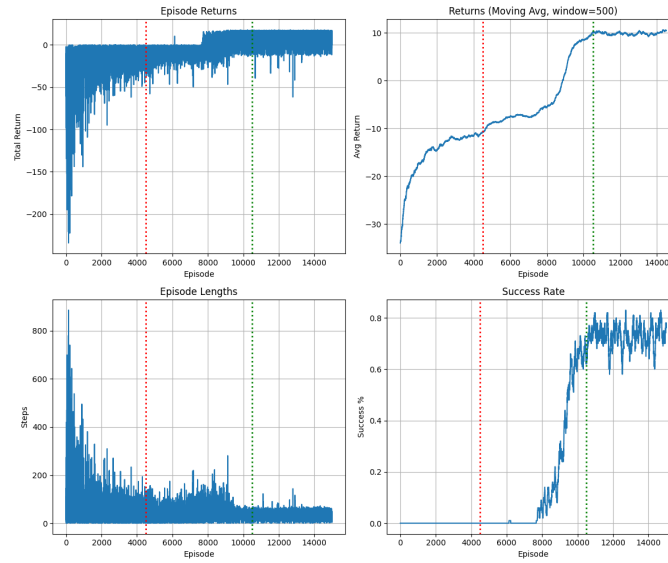


Figura 4: Métricas de entrenamiento en Entorno 2: retornos, media móvil, longitud de episodios y tasa de éxito.

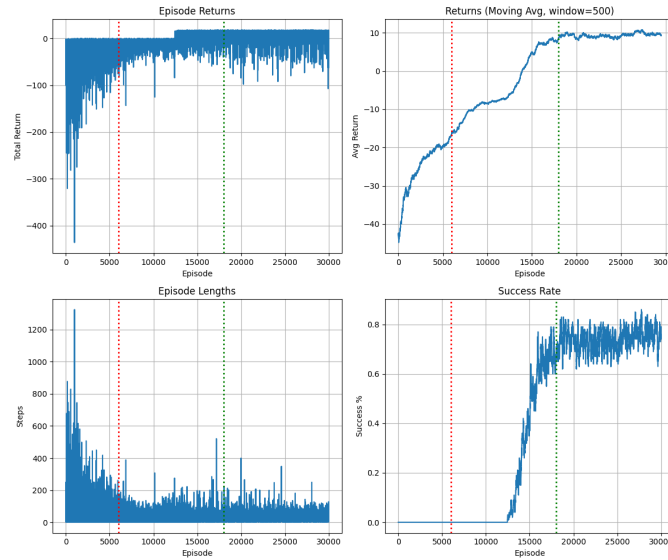


Figura 5: Métricas de entrenamiento en Entorno 3: retornos, media móvil, longitud de episodios y tasa de éxito.

2.8. Conclusión

La combinación de SARSA(λ) y *tile coding* permite resolver de forma robusta los entornos de almacén, tanto en tareas de recogida como de entrega, incluso cuando la posición de los objetos varía aleatoriamente. La representación vectorizada no resulta adecuada en estos casos, mientras que los mosaicos solapados facilitan la generalización y el aprendizaje eficiente.